J-FALL 2026

# Context Is a Budget

Eight levers and three workflow patterns for spending tokens well with coding agents.

github.com/sohamda/ai-token-management

ONE FEATURE, TWO WAYS

# Same feature. Same repo. Very different bill.

When an order total exceeds $100, the confirmation email adds "VIP customer — free shipping on your next order".

### The default way

- Opus at max effort, one long session
- "Add a VIP line to the order emails"
- Raw `mvn test` output, every run

**$?.??**

MEASURED ON THIS REPO

### The way this talk ends

- Sonnet session, Opus only to plan and review
- Scoped asks, fresh context per role
- Test output trimmed to the failures

**$?.??**

MEASURED ON THIS REPO

Every step between those two numbers is one of eight levers or three patterns.

SET THE STAGE

# The budget is real now

### Copilot · since June 1

**AI Credits** bill every input, output and cached token at the model's API rate. Premium requests are gone.

### Claude Code

**Per token** on the API. On a subscription: an allowance that resets every **5 hours** and every **week**.

**~$13**

PER DEVELOPER PER ACTIVE DAY

Anthropic's enterprise average; 90% of users stay under $30.



### Cost

An invoice, or the agent stopping for the afternoon.



### Speed

Every token in the window is processed on every turn.



### Quality

Recall and reasoning **degrade** as the window fills.

KNOW YOUR ENEMY

# Anatomy of a request

| LAYER | WHAT MAKES IT GROW |
| --- | --- |
| System prompt + tools | MCP servers, plugins. Small now that tools load on demand |
| Project instructions | `AGENTS.md`, `CLAUDE.md`, unscoped rules |
| Files and tool output | Broad exploration, whole-file reads, **build and test logs** |
| Conversation history | Every turn you don't clear or compact |
| Output and thinking | Narration, recaps, high effort on easy tasks: **5× the input price** |

All of it is re-sent on **every turn**. Caching makes that cheap, not free.

ONE YEAR LATER

# The tool does the mechanics. You own the decisions.

| A YEAR AGO, YOU HAD TO… | NOW THE TOOL DOES IT | STILL YOUR CALL |
|---|---|---|
| Order prompts so they cache | Cache breakpoints placed for you | **Not breaking** the cache |
| Prune MCP tool schemas | Tools load on demand | What tools **return** |
| Summarise chats by hand | Auto-compaction | **When** to clear or compact |
| Guess, because tokens were invisible | `/context`, `/usage`, status line | **Looking**, and setting a cap |
| Dodge Copilot's "Opus = 10×" | Per-token billing | Model **and effort** per task |

ROADMAP

# Eight levers, three patterns

A
**Scope the ask**

B
**Don't break the cache**

C
**Make tool output cheap**

D
**Codify once, load on demand**

E
**Right-size model and effort**

F
**Output discipline**

G
**Make the repo cheap to read**

H
**Read the meter, set a budget**

1
**Loop with state on disk**
many small tasks

2
**Checkpoint & reset**
one long task

3
**Plan → Build → Review**
one feature, several roles

LEVER A · DEMO 01

# Scope the ask

The agent can't ask what you meant until it has explored. Exploration is what you pay for.

## Vague

```
Refactor the order confirmation email
to use the new template engine.
```

Finds the Java **and** a dead JavaScript copy, reads both, then asks which one you meant.

## Scoped

```
In @…/OrderConfirmationService.java,
call render("order_confirmation_v2", …)
instead of renderLegacy.
Keep behaviour identical.
```

Two files read. One small diff.

LEVER B · DEMO 02

# Don't break the cache

Every turn re-sends the whole conversation. Cached, it costs **10%** of input on Sonnet, **5%** on Opus.

## Breaks it

- **Switching model** mid-session
- Coming back after it expires: **1 h** on a subscription, **5 min** on an API key
- Compaction, by design

## Cache-safe

- Editing files, skills, plan mode, subagents
- `/effort` on the 5.5 models
- Editing `AGENTS.md`, but it applies only after `/clear`

LEVER C · DEMO 03

# Make tool output cheap

Tool definitions load on demand now. The tax moved to what tools **return**.

## Filter before the model sees it

```
# .claude/hooks/filter-test-output.sh
mvn test  →  mvn test > $log 2>&1
             grep -E 'Tests run:|FAILURE|BUILD'
             exit $rc
```

??? lines → ?? lines. Red still reads as red.

## Other ways

- **Subagents** for noisy work: only the summary comes back
- **CLIs over MCP**: `gh pr view 42`
- **Your own tools**: `{id, summary}` by default

LEVER D

# Codify once, load on demand

| LOADS | CLAUDE CODE | COPILOT |
|---|---|---|
| Every session | `CLAUDE.md` or `AGENTS.md` | `copilot-instructions.md` |
| Matching file touched | `.claude/rules/` + `paths:` | `*.instructions.md` + `applyTo:` |
| When invoked | Skills, subagents | Prompt files, skills |

### Keep the always-on layer short

Under 200 lines. This repo's `AGENTS.md` is 30.

### Two gotchas

- Test rules scoped to `src/**` load on every production edit.
- A `CLAUDE.md` hides `AGENTS.md` unless it imports `@AGENTS.md`.

LEVER E

# Right-size the model *and* the effort

| MODEL | IN / OUT PER MTOK |
|---|---|
| Haiku 5.5 | $0.10 / $0.50 |
| Sonnet 5.5 | $2 / $10 |
| Opus 5.5 | $4 / $20 |

Opus is 2× Sonnet now. Sonnet is 20× Haiku.

## Effort is the cheaper knob

- The 5.5 models always think; **thinking bills as output**
- Changing effort keeps the cache. Changing model doesn't.
- Default: Sonnet · medium. Haiku for subagents.

LEVER F

# Output discipline

Output costs **5× input**, then gets re-read on every later turn.

### Paragraphs and a tour

```
Explain TemplateEngine.java.
```

### Three lines

```
In three bullets: what does
TemplateEngine.java do, and which
method should new emails call?
```

### Make it the default, in AGENTS.md

```
- Be concise. No preamble.
- Return diffs, not full files.
- Ask one targeted question instead of guessing.
```

LEVER G

# Make the repo cheap to read

Claude Code greps; Copilot searches an index. Either way, dead code gets found.

## This repo's trap

An old JavaScript copy of the email code. Every search for "order confirmation" finds it, and the agent reads it.

## Fix it

- **Delete dead code.** Git keeps it.
- Deny what you can't delete: `Read(./target/**)`
- Three-line summary at the top of each class

LEVER H · DEMO 14

# Read the meter, set a budget

```
Sonnet 5.5 · medium  |  ctx 23%  |  cache 91%  |  $0.46  |  5h 12%
```

## Look

- Status line, always on
- `/context`: what's in the window
- `/usage`: cost, cache, what used it

## Cap

- `--max-budget-usd` on every headless run
- Team spend limits
- `npm run lint:context` in CI

FOR COPILOT USERS

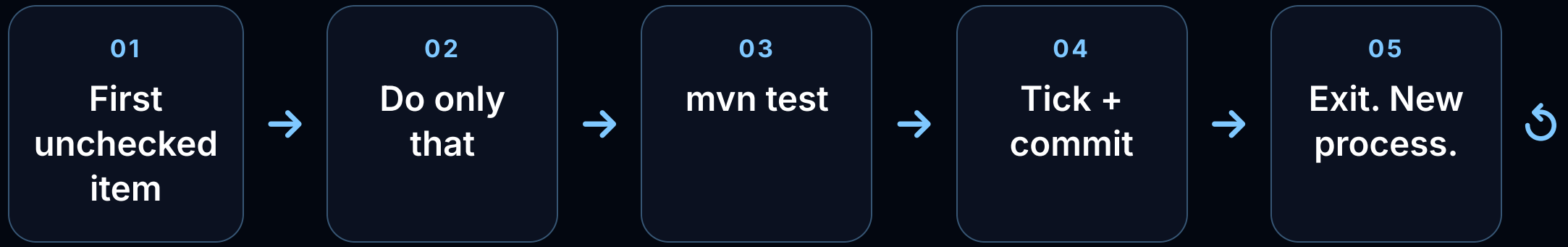
# Same levers, your tool

| | CLAUDE CODE | COPILOT |
|---|---|---|
| **A** | `@path`, plan mode | `#file`, `#selection` |
| **B** | cache line in `/usage` | Auto switches only at session boundaries |
| **C** | hooks, subagents, `/mcp` | tool search; filter what commands print |
| **D** | `AGENTS.md`, `.claude/rules/`, skills | `copilot-instructions.md`, `applyTo`, prompt files |
| **E** | `/model`, `/effort`, per-subagent model | model picker, reasoning level, Auto |
| **G** | deny rules, `.gitignore` | index follows `.gitignore`, content exclusions |
| **H** | status line, `/context`, `/usage` | context indicator, usage page, AI-credit budgets |
| **1–3** | `claude -p` loop, `/clear`, subagents | cloud agent per issue, new chat, prompt files |

Copilot column from GitHub and VS Code docs. Check what your IDE's plugin supports.

PATTERN 1 · DEMO 08

# Loop with state on disk (Ralph)




## Not this

"Repeat until TODO.md is done" **inside one chat**. Item five carries items one to four.

## This: `tools/ralph.sh`

```
while grep -q '^- \[ \]' TODO.md; do
  claude -p "$(cat tools/ralph-prompt.md)" \
    --model sonnet --max-budget-usd 0.50
done
```

PATTERN 2

# Checkpoint & reset

Compaction is automatic. **When** you reset is your call, and the options cost very different amounts.

| ACTION | COST | USE IT WHEN |
|---|---|---|
| `/clear` | **Free** | Switching to an unrelated task |
| `/rewind` | Small: reuses the cache | The last turns went the wrong way |
| `/compact <focus>` | Reads the whole conversation | Mid-task, continuing with less |
| Auto-compaction | Same, at a moment you didn't choose | You forgot the other three |

PATTERN 3 · DEMO 10

# Plan → Build → Review

**planner**
opus · high · writes plan.md

→

**implementer**
sonnet · medium · commit per task

→

**reviewer**
opus · high · verdict

## Why it's cheaper

- Each subagent **starts fresh**
- Only summaries come back
- Your session **keeps its model and cache**

## Fan-out isn't free

- Each subagent pays for its own context
- Agent teams: **~7× the tokens**
- Pass artifacts, never chat history

FOR A JAVA AUDIENCE

# Java's token sinks

Java's build tools are chatty, and every line they print becomes context.

| SINK | FIX |
| --- | --- |
| Maven / Gradle output | `mvn -B -ntp -q` · `gradle -q --console=plain` · or a hook |
| Spring Boot startup logs | `spring.main.banner-mode=off`, `logging.level.root=warn` in a dev profile |
| Deep stack traces | Keep frames from your own packages; drop proxy and reflection frames |
| Generated sources | Deny `target/generated-sources/**` (Lombok, MapStruct, OpenAPI) |
| Multi-module builds | `mvn -pl <module> -am test`, `-Dtest=<Class>Test` |

BACK TO THE FEATURE

# Where the difference came from

| | DEFAULT WAY | DISCIPLINED WAY | LEVER |
|---|---|---|---|
| The ask | "Add a VIP line" | planner writes plan.md first | A · 3 |
| Model | Opus · max, throughout | Sonnet session, Opus only to plan and review | E · B |
| Test output | full Maven log, every run | summary and failures | C |
| Context | one session, growing | fresh context per role | 2 · 3 |
| **Cost** | **$?.??** | **$?.??** | **H** |

TAKE-HOME

# If you only do three things

1

**`/clear` between tasks, and name the file**

Free, and it works tomorrow morning.

2

**Trim what your build prints**

One flag or one hook pays off on every test run.

3

**Turn on the meter**

A status line with context %, cache % and cost.

Full Monday checklist in the repo and the appendix.

IN ONE SENTENCE

# The tool handles the mechanics. You own the decisions.



github.com/sohamda/ai-token-management

slides · demos · blog · .claude config

Questions?



APPENDIX

# For questions

Calculator · anti-patterns · scaling up · Monday checklist · references

APPENDIX · CALCULATOR

# What does this actually cost?

Model

Claude Sonnet 5.5 ($2 / $10)

Input tokens / request

60000

Output + thinking tokens / request

1500

Requests / day / dev

200

Developers

50

Prompt cache hit rate

0% | 50% | 90% | 95%

| | |
|---|---|
| Per request | $0.038 |
| Per dev / day | $7.560 |
| Per dev / month (22d) | $166.32 |
| Team / month | $8,316 |

At 90% cache hit rate you save $21,384/mo (72%) vs no caching. For scale: Anthropic reports ~$13 per developer per active day for Claude Code.

Claude API list prices (Oct 2026). Cache reads: 10% of input (Haiku, Sonnet), 5% (Opus 5.5), 2.5% (Fable 5.1); cache writes ignored. Copilot bills these same API rates since June 1.

CALL THESE OUT IN REVIEW

# Anti-patterns to stamp out

## What goes in

- One session for the whole day's unrelated tasks.
- Vague asks in a repo full of dead code.
- Raw `mvn test` logs and full stack traces in context.
- A new `CLAUDE.md` that silently hides `AGENTS.md`.
- `**` globs on scoped rules.

## How it runs

- `/model` mid-task "just to check".
- Opus at max effort for a rename.
- A "loop" that lives inside one chat; `/loop` for batch work.
- Five subagents where one would do.
- Headless runs without `--max-budget-usd`.

APPENDIX · SCALING UP (DEMOS 11–13)

# From one repo to a fleet of agents

Adapted from a production repo running ~40 specialised skills. Each layer is portable on its own.

| # | LAYER | WHAT IT GIVES YOU | IN CLAUDE CODE |
|---|---|---|---|
| 1 | **Budget at design time** | Know the fixed cost before writing the agent | `/context` in a fresh session |
| 2 | **Progressive disclosure** | AGENTS.md → rules → skills → references | `.claude/rules/`, skills |
| 3 | **Context shredding** | Full / digest / minimal variants of each skill | pick by `ctx %` |
| 4 | **Compaction phases** | Agents summarise at 60–80% instead of drowning | `# Compact instructions` |
| 5 | **Subagent extraction** | Heavy phases run in a fresh context, return a result | `.claude/agents/` |
| 6 | **Glob targeting** | No rule loads everywhere by accident | `paths:` + `lint:glob-audit` |
| 7 | **Observability** | Measure, then lint for regressions in CI | `/usage`, OpenTelemetry, `lint:context` |

APPENDIX · TAKE-HOME

# The Monday checklist

## This week

- [ ] One always-on instructions file under 200 lines: commands, conventions, output style.
- [ ] Both `AGENTS.md` and `CLAUDE.md`? Import with `@AGENTS.md`.
- [ ] A status line with context %, cache % and cost.
- [ ] A hook that trims test and build output.
- [ ] Delete dead code. Deny generated folders.
- [ ] Model at session start; `/clear` between tasks.

## This month

- [ ] Scoped rules with narrow globs; workflows into skills and subagents.
- [ ] Haiku subagents for search and summaries.
- [ ] A headless Ralph loop over a backlog of chores.
- [ ] Planner → implementer → reviewer on one real feature.
- [ ] `--max-budget-usd` on every headless run; spend limits for the team.
- [ ] `npm run lint:context` in CI.

APPENDIX · FURTHER READING

# References

## Context and cost

- Anthropic — Effective context engineering for AI agents
- Claude Code — Manage costs effectively
- Claude Code — How Claude Code uses prompt caching
- Chroma — Context Rot
- Claude — Pricing
- VS Code — Improving token efficiency in GitHub Copilot

## Patterns and tools

- Geoffrey Huntley — Ralph Wiggum as a "software engineer"
- Claude Code — Subagents · Hooks · Memory & AGENTS.md
- Cognition — Don't build multi-agents
- GitHub — Copilot is moving to usage-based billing
- GitHub — Getting more from each token